

Week 4. Homework 2. SQL-II

1. Find the First and Last Movie Released Each Year
 - Retrieve the first and last movie released in each year. Display the year, first movie title, and last movie title.

Code:

```
SELECT strftime('%Y', release_date) AS Year,  
min(title) AS first_movie,  
max(title) AS last_movie  
FROM movies  
GROUP BY Year  
ORDER BY Year;
```

Screenshot:

	Year	first_movie	last_movie
1	1916	Intolerance	Intolerance
2	1925	The Big Parade	The Big Parade
3	1927	Metropolis	Metropolis
4	1929	Pandora's Box	The Broadway Melody
5	1930	Hell's Angels	Hell's Angels
6	1932	A Farewell to Arms	A Farewell to Arms
7	1933	42nd Street	She Done Him Wrong
8	1934	It Happened One Night	It Happened One Night

```
Execution finished without errors.  
Result: 90 rows returned in 15ms  
At line 1:  
SELECT strftime('%Y', release_date) AS Year,  
min(title) AS first_movie,  
max(title) AS last_movie  
FROM movies  
GROUP BY Year  
ORDER BY Year;
```

2. Total Revenue Generated Each Year
 - Calculate the total revenue generated by movies released in each year. Display the year and total revenue, ordered by revenue in descending order.

Code:

```
SELECT strftime('%Y', release_date) AS Year,  
SUM(revenue) as total_revenue  
FROM movies
```

GROUP BY Year

ORDER BY total_revenue DESC;

Screenshot:

	Year	total_revenue
1	2012	24141710246
2	2014	24120490589
3	2013	23411493295
4	2015	22775024221
5	2009	21072651506
6	2011	20516921160
7	2010	20348574768
8	2008	18145379803

Execution finished without errors.
Result: 90 rows returned in 10ms
At line 9:
SELECT strftime('%Y', release_date) AS Year,
SUM(revenue) as total_revenue
FROM movies
GROUP BY Year
ORDER BY total_revenue DESC;

3. Movies Released in the First Half of the Year

- Find all movies that were released between January and June. Display the movie title, release date, and director name.

Code:

```
SELECT DISTINCT movies.title, movies.release_date, directors.name
```

```
FROM movies
```

```
JOIN directors ON movies.director_id = directors.id
```

```
WHERE strftime('%m', movies.release_date) BETWEEN '01' AND '06';
```

Screenshot:

	title	release_date	name
1	Pirates of the Caribbean: At World's End	2007-05-19	Gore Verbinski
2	John Carter	2012-03-07	Andrew Stanton
3	Spider-Man 3	2007-05-01	Sam Raimi
4	Avengers: Age of Ultron	2015-04-22	Joss Whedon
5	Batman v Superman: Dawn of Justice	2016-03-23	Zack Snyder
6	Superman Returns	2006-06-28	Bryan Singer
7	Pirates of the Caribbean: Dead Man's Chest	2006-06-20	Gore Verbinski
8	Man of Steel	2013-06-12	Zack Snyder

Execution finished without errors.
Result: 2166 rows returned in 17ms
At line 16:
SELECT DISTINCT movies.title, movies.release_date, directors.name
FROM movies
JOIN directors ON movies.director_id = directors.id
WHERE strftime('%m', movies.release_date) BETWEEN '01' AND '06';

4. Number of Movies Released on Weekends

- Count the total number of movies released on a weekend (Saturday or Sunday). Use SQLite's strftime('%w') to determine the day of the week (0 = Sunday, 6 = Saturday).

Code:

```
SELECT strftime('%w', release_date) AS Weekend,
```

```
COUNT(*) as Num_movies
```

```
FROM movies
```

```
WHERE strftime('%w', release_date) = '0' OR strftime('%w', release_date) = '6';
```

Screenshot:

	Weekend	Num_movies
1	6	451

Execution finished without errors.
Result: 1 rows returned in 12ms
At line 22:
SELECT strftime('%w', release_date) AS Weekend,
COUNT(*) as Num_movies
FROM movies
WHERE strftime('%w', release_date) = '0' OR strftime('%w', release_date) = '6';

5. Categorize Directors Based on Total Revenue

- Classify directors based on the total revenue of their movies:

- High Revenue if total revenue > \$1 billion
- Moderate Revenue if total revenue is between \$500 million and \$1 billion
- Low Revenue if total revenue < \$500 million

Code:

```
SELECT DISTINCT name, total_revenue,
CASE
WHEN total_revenue >= 1000000000 THEN 'High Revenue'
WHEN total_revenue >= 500000000 THEN 'Moderate Revenue'
ELSE 'Low Revenue' END AS Categorize_Directors
FROM ( SELECT directors.name,
SUM(movies.revenue) AS total_revenue
FROM movies
JOIN directors ON movies.director_id = directors.id
GROUP BY directors.name
);
```

Screenshot:

	name	total_revenue	Categorize_Directors
1	Aaron Hann	0	Low Revenue
2	Aaron Schneider	0	Low Revenue
3	Abel Ferrara	1227324	Low Revenue
4	Adam Brooks	55447968	Low Revenue
5	Adam Carolla	0	Low Revenue
6	Adam Goldberg	0	Low Revenue
7	Adam Green	0	Low Revenue
8	Adam Jay Epstein	0	Low Revenue

```

Execution finished without errors.
Result: 2349 rows returned in 23ms
At line 28:
SELECT DISTINCT name, total_revenue,
CASE
WHEN total_revenue >= 1000000000 THEN 'High Revenue'
WHEN total_revenue >= 500000000 THEN 'Moderate Revenue'
ELSE 'Low Revenue' END AS Categorize_Directors
FROM ( SELECT directors.name,
SUM(movies.revenue) AS total_revenue
FROM movies
JOIN directors ON movies.director_id = directors.id
GROUP BY directors.name
);

```

6. Movies Based on Popularity and Revenue

- Classify movies into the following categories based on popularity and revenue:

- Blockbuster if popularity > 100 and revenue > \$500 million
- Hit if popularity > 50 and revenue between \$100 million and \$500 million
- Average for all other movies

Code:

```
SELECT title, revenue,
```

```
CASE
```

```
WHEN popularity > 100 AND revenue > 500000000 THEN 'Blockbuster'
```

```
WHEN popularity > 50 AND revenue > 100000000 AND revenue < 500000000 THEN 'Hit'
```

```
ELSE 'Average' END AS Movies_Pop_Rev
```

```
FROM movies;
```

Screenshot:

	title	revenue	Movies_Pop_Rev
1	Avatar	2787965087	Blockbuster
2	Pirates of the Caribbean: At World'...	961000000	Blockbuster
3	Spectre	880674609	Blockbuster
4	The Dark Knight Rises	1084939099	Blockbuster
5	John Carter	284139100	Average
6	Spider-Man 3	890871626	Blockbuster
7	Tangled	591794936	Average
8	Avengers: Age of Ultron	1405403694	Blockbuster

```

Execution finished without errors.
Result: 4773 rows returned in 12ms
At line 41:
SELECT title, revenue,
CASE
WHEN popularity > 100 AND revenue > 500000000 THEN 'Blockbuster'
WHEN popularity > 50 AND revenue > 100000000 AND revenue < 500000000
THEN 'Hit'
ELSE 'Average' END AS Movies_Pop_Rev
FROM movies;

```

7. Directors with Critically Acclaimed and Poorly Rated Movies

- List directors and count how many of their movies fall into these categories:
 - Critically Acclaimed if vote_average >= 8
 - Poorly Rated if vote_average < 5

Code:

```

SELECT name, Directors_rating,
COUNT(*) AS Num_movies
FROM (
SELECT directors.name,
CASE
WHEN vote_average >= 8 THEN 'Critically Acclaimed'
WHEN vote_average < 5 THEN 'Poorly Rated'
ELSE 'Average' END AS Directors_rating
FROM movies
JOIN directors ON movies.director_id = directors.id)
GROUP BY name, Directors_rating;

```

Screenshot:

	name	Directors_rating	Num_movies
1	Aaron Hann	Average	1
2	Aaron Schneider	Average	1
3	Abel Ferrara	Average	1
4	Adam Brooks	Average	1
5	Adam Carolla	Average	1
6	Adam Goldberg	Average	1
7	Adam Green	Average	1
8	Adam Jay Epstein	Poorly Rated	1

```

Execution finished without errors.
Result: 2537 rows returned in 14ms
At line 49:
SELECT name, Directors_rating,
COUNT(*) AS Num_movies
FROM (
SELECT directors.name,
CASE
WHEN vote_average >= 8 THEN 'Critically Acclaimed'
WHEN vote_average < 5 THEN 'Poorly Rated'
ELSE 'Average' END AS Directors_rating
FROM movies
JOIN directors ON movies.director_id = directors.id)
GROUP BY name, Directors_rating;

```

AI Usage: I did use AI to get guidance on Questions 3, 5, and 7. I did attempt it on my own, but I am still struggling a little with the JOIN statement. Other than those three questions, I have attempted all the questions.